

DRAGON 9 ENIGMA PIPELINE

SYSTEM RESET & TECHNICAL BRIEF: DAWNENGINE SCENIC AUTOMATION

Project Node: Enigma Pipeline / DawnEngine v2 Integration
Target Engine: Kdenlive (MLT XML Engine) + Kimi-Audio QC
Media Asset Pool: 17,241 indexed files (1.71 TB Total Footprint)
Status: READY FOR EXECUTION

1. Executive Overview & Architecture Rethink

This technical brief establishes the redesigned automation pipeline for creating scenic B-roll edits synchronized to DawnEngine procedural audio. By bypassing previous directory mismatches and missing-file errors, the system now validates paths directly against the `audit_inventory.json` catalog on disk before timeline construction.

2. DawnEngine Audio Generation Specifications

- **Genre Profile:** Ambient Organic / Dub Roots / Scenic Coastal Downtempo
- **Tempo & Meter:** 90 BPM (Quantized 4/4 grid with transient hit markers)
- **Acoustic Profile:** Warm harmonic sub-frequencies, rhythmic percussion transients, dynamic range optimized for coastal visual montage.
- **Output Destination:** `/home/tim/work/enigma-pipeline/storage/dawnengine_ambient_90bpm.wav`

3. Automated Pipeline Integration Workflow

To execute the clean reboot without manual path hunting, execute the unified pipeline controller script below:

```
cat << 'EOF' > /home/tim/work/enigma-pipeline/pipeline_controller.py
import json, os, random
import xml.etree.ElementTree as ET

AUDIT_JSON = "/home/tim/work/enigma-pipeline/storage/audit_inventory.json"
OUTPUT_KDENLIVE = "/home/tim/work/enigma-pipeline/scenic_dawnengine_edit_v2.kdenlive"
AUDIO_PATH = "/home/tim/work/enigma-pipeline/storage/dawnengine_ambient_90bpm.wav"

def run_pipeline():
    print("🔧 Initializing DawnEngine Pipeline v2 Reset...")
    if not os.path.exists(AUDIT_JSON):
        print(f"❌ Missing catalog: {AUDIT_JSON}")
        return

    with open(AUDIT_JSON, "r") as f:
        data = json.load(f)

    # Recursive path extractor
    all_paths = []
    def extract(obj):
```

```

        if isinstance(obj, dict):
            for k, v in obj.items():
                if isinstance(v, (dict, list)): extract(v)
                elif isinstance(v, str) and os.path.exists(v): all_paths.append(v)
        elif isinstance(obj, list):
            for i in obj: extract(i)

extract(data)

video_clips = [p for p in set(all_paths) if p.lower().endswith(('.mp4', '.mov', '.avi'))]
print(f"✅ Verified {len(video_clips)} active video clips on disk.")

root = ET.Element("mlt", LC_NUMERIC="C", version="7.0.0", title="DawnEngine Scenic Reset")
ET.SubElement(root, "profile", {"description": "HD 1080p 25fps", "frame_rate_num": "25",
"frame_rate_den": "1", "width": "1920", "height": "1080", "progressive": "1"})

tractor = ET.SubElement(root, "tractor", id="maintractor")
multitrack = ET.SubElement(tractor, "multitrack")
ET.SubElement(multitrack, "track", producer="playlist_video")
ET.SubElement(multitrack, "track", producer="playlist_audio")

playlist_v = ET.SubElement(root, "playlist", id="playlist_video")
playlist_a = ET.SubElement(root, "playlist", id="playlist_audio")

# Lock Audio
ET.SubElement(root, "chain", id="prod_audio", src=AUDIO_PATH)
ET.SubElement(playlist_a, "entry", producer="prod_audio", in_="0", out="90000")

# Map Video Cuts (4s cuts / 100 frames at 25fps)
selected = random.sample(video_clips, min(40, len(video_clips)))
for idx, clip in enumerate(selected):
    pid = f"prod_v_{idx}"
    ET.SubElement(root, "chain", id=pid, src=clip)
    ET.SubElement(playlist_v, "entry", producer=pid, in_="0", out="100")

tree = ET.ElementTree(root)
ET.indent(tree, space=" ")
tree.write(OUTPUT_KDENLIVE, encoding="utf-8", xml_declaration=True)
print(f"🎉 New Timeline Created: {OUTPUT_KDENLIVE}")

if __name__ == "__main__":
    run_pipeline()
EOF
python3 /home/tim/work/enigma-pipeline/pipeline_controller.py

```

4. Immediate Action Plan

- Run the pipeline controller script above inside your terminal.
- Launch Kdenlive directly: `kdenlive /home/tim/work/enigma-pipeline/scenic_dawnengine_edit_v2.kdenlive &`